

JEV + CLAUDE OPUS 5.5 BEST STACK FOR A COMPANY BRAIN

A cheap decision layer wired in front of an expensive reasoning model,
and the arithmetic that decides when it actually pays

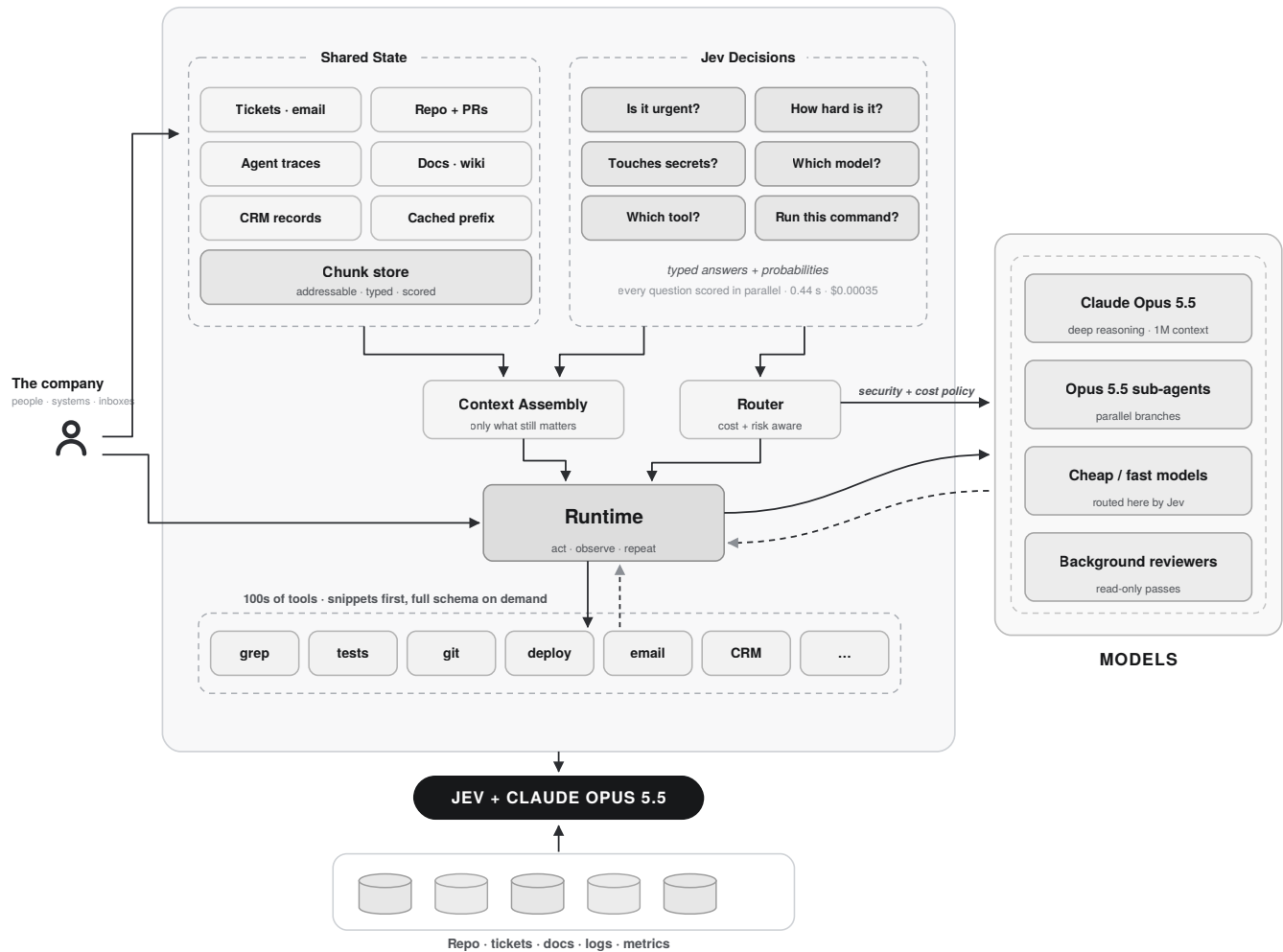


Fig. 1. A company brain. Shared state and a bank of typed Jev questions feed context assembly and a router; the runtime acts through tools and calls a model only when the decision layer says it is worth it.

Abstract— Two releases in September 2026 changed the shape of a production agent. TypeSafe AI shipped *Jev*, a "System One" model that does not generate text at all: it takes a state and a set of typed questions and returns answers with calibrated probabilities, reportedly up to 200x faster and 400x cheaper than a comparable LLM on classification work. Anthropic shipped *Claude Opus 5.5*

at \$4 / \$20 per million tokens with a 1M-token context window, 66.4% on Terminal-Bench 4.0, and roughly 40% lower cost per task than Opus 5. Read separately, they are two model launches. Read together, they describe an architecture: a cheap decision layer wired in front of an expensive reasoning layer, with ordinary code holding the branch. This paper works through that

architecture end to end — the decision primitive, the reasoning tier, the router, the trust boundary, the evaluation loop, and the arithmetic that decides when routing actually saves money. It closes with a five-week rollout that never replaces a working call before the numbers agree, and a list of the ways this design fails.

Index Terms— agentic systems, System One models, typed decisions, calibrated probabilities, model routing, tool-call gating, LLM-as-judge, online evaluation, context compression, agent harness.

I. INTRODUCTION

A. The bottleneck moved

For three years the interesting question about a language model was whether it could produce the answer at all. That question is largely settled for the work most companies actually have. A frontier model can read a two-hundred-thousand-line codebase, plan a migration, write the diff and explain the tradeoff. The constraint is no longer capability. It is that every single step of an agent loop currently costs a full model call, and most of those steps are not thinking. They are decisions.

Consider what an agent does between the moment a request arrives and the moment something ships. It decides whether the request is urgent. It decides how hard it is. It decides which tool to reach for. It decides whether the tool output was useful. It decides whether the context it is carrying is still relevant. It decides whether the command it is about to run is safe. Each of those is a small, bounded, repeatable judgement with a finite answer space. Each of them, today, is answered

by asking a text generator to produce tokens that some downstream parser turns back into a value.

That is an expensive way to get a boolean. It is also an unreliable one: the same trace scored twice by the same judge does not necessarily produce the same number.

B. What this document is

This is a synthesis, not an announcement. It reads the public material on Jev (the LangChain integration post and the LangChain evaluation experiment), the public material on Claude Opus 5.5 (the September 2026 release and third-party benchmark write-ups), and a widely circulated engineering note from TypeSafe on what a typesafe-centric coding agent would look like. It then does the part none of those sources do on their own: it puts the two models in one architecture and asks what breaks.

Every number in this paper is attributed. Vendor claims are labelled as vendor claims. Figures derived by arithmetic from published per-unit prices are labelled as derived. Nothing here should be read as a benchmark we ran.

II. TWO SYSTEMS, ONE LOOP

The useful frame is the one TypeSafe uses itself. A System One model is built to make fast, structured decisions that software can consume directly. It evaluates a state and returns typed answers and probabilities. It is trained with reinforcement learning for calibrated decisions. It does not write prose, and that is the point: the output is already the value your code needs.

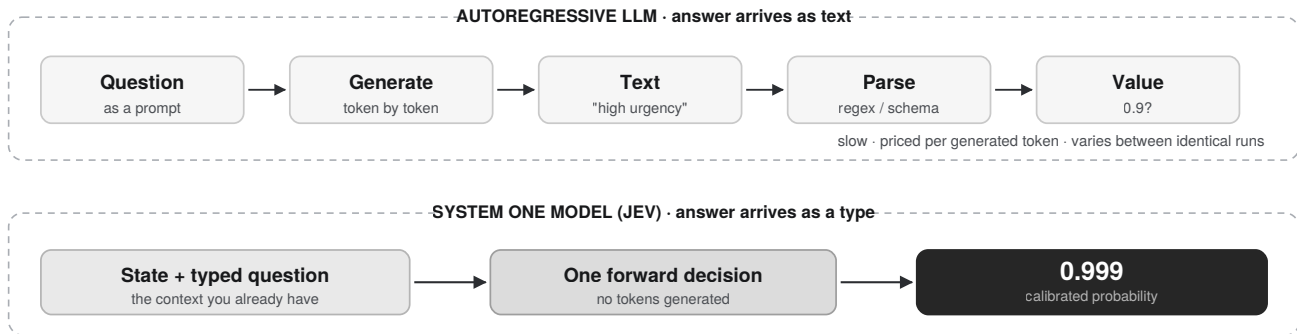


Fig. 2. The same question, answered two ways. An autoregressive model reaches a judgement by generating tokens that then have to be parsed back into a value. A System One model returns the value.

An autoregressive model takes the long road to a short answer. It receives a prompt, generates a sequence, emits text that says something like *high urgency*, and then a schema or a regular expression converts that back into a number your branch can use. Three things follow. It is slow, because tokens are produced one at a time. It is priced on generated tokens, so the cost of the decision scales with how verbose the model feels like being. And it is not repeatable, because sampling is stochastic and the mapping from text to score is lossy.

A System One model collapses that path. State plus typed question in; typed answer plus probability out. The probability is not a formatting convention that the model was asked to emit — it is the model's calibrated belief. Your code reads it as a float and branches on it.

TABLE I
THE TWO TIERS OF A COMPANY BRAIN

Property	System One (Jev)	System Two (Opus 5.5)
Output	typed answer + probability	text, code, diffs, plans
Unit of work	one bounded decision	one open-ended task
Latency, observed	0.44 s average per call	seconds to hours per task
Cost, observed / listed	\$0.00035 per call	\$4 in / \$20 out per Mtok
Repeatability	lowest observed variance	sampling-dependent
Parallelism	all questions in one request	sequential turns
Good at	triage, routing, gating, scoring	planning, writing, explaining
Bad at	anything open-ended	being called ten thousand times an hour

Sources: LangChain's Jev integration post and evaluation experiment; Anthropic's Opus 5.5 release pricing. Latency and per-call cost are the averages LangChain observed in a five-example weather-agent experiment, not a general benchmark.

The two tiers are not competitors. They are different organs. The mistake worth avoiding is the reflex to treat Jev as a small LLM you can also chat with. It has no chat mode. It is closer in spirit to a classifier you did not have to train than to a model you prompt.

III. INSIDE JEV: THE DECISION PRIMITIVE

A Jev request has two parts: a *state* and a set of *questions*. The state is the context you already have — a ticket body, a diff, a tool result, a trace, a proposed command. The questions are typed, and there are exactly three shapes.

A. Three question types

TABLE II
QUESTION TYPES AND WHAT THEY RETURN

Type	Question shape	Returns
Choice	pick one option from a set	probability per option, plus a confidence
Score	rate against an ordered rubric	a continuous score, the distribution, a confidence
Noul	is this statement true	the probability that it is true, 0.0 to 1.0

The vocabulary is small on purpose. Almost every decision inside an agent loop is one of these three. *Which tool* is a Choice. *How hard is this* is a Score. *Does this command touch a credential* is a Noul. Once you have noticed that, you start seeing chat calls in your codebase that were never questions at all.

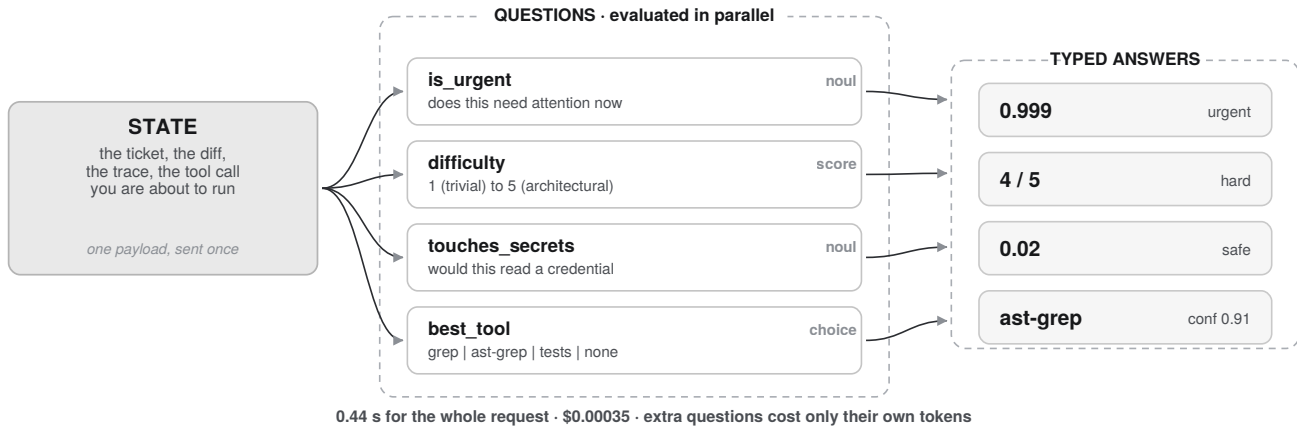


Fig. 3. Anatomy of a single Jev call. One state, four typed questions, four typed answers. The questions are evaluated in parallel, so the fourth question is close to free.

B. Everything in one request, in parallel

This is the property that changes how you design. System One models evaluate every question in a request in parallel. Adding questions barely moves the response time and costs only the tokens for the extra questions, which are cheap. The economics of asking invert. With a chat model you ask the fewest questions you can get away with, because each one is a round trip. With Jev

you ask everything you might want to know, because the marginal question is nearly free and the answer you did not ask for is the one you needed.

THE DESIGN SHIFT. Stop writing one prompt that asks a model to reason about a situation and return JSON. Start writing a list of small, atomic, independently useful questions, and let them all be answered at once.

C. Calling it

The raw request is a state and a dictionary of questions. Here it is with a single Noul, in the shape `TypeSafe` documents:

```
{
  "model": "jev-latest",
  "state": "Deploy failed twice, users
    see 500s. Look now?",
  "questions": {
    "is_urgent": {
      "type": "noul",
      "instructions": "Conveys urgency"
    }
  }
}
```

And the answer, which is already the value:

```
{ "is_urgent": { "type": "noul", "noul":
0.999 } }
```

`LangChain` exposes the same thing through `TypeSafeClassifier`. You pass state and questions to `.invoke()` and get classification results rather than a chat response. The state can be text, structured data, or `LangChain` messages, which means you can hand it the context your agent is already carrying.

```
from langchain_typesafe import (
    Noul, Score, TypeSafeClassifier)

clf = TypeSafeClassifier()

r = clf.invoke({
  "state": incoming,
  "questions": {
    "urgent": Noul(
      instructions="Needs attention now?"),
    "difficulty": Score(
      instructions="1 trivial, 5 huge"),
    "risky": Noul(
      instructions="Reads a credential?"),
  },
})

if r.nouls["risky"].noul > 0.15:
    return refuse(incoming)
if r.scores["difficulty"].score >= 4:
    return escalate(incoming) # Opus 5.5
return fast_path(incoming)
```

Read the last four lines carefully, because they are the whole thesis. There is no second model call in the common branch. The probability is the branch condition. The decision has been moved out of a prompt and into ordinary, reviewable, testable control flow.

IV. INSIDE CLAUDE OPUS 5.5

The other half of the stack moved at the same time. Claude Opus 5.5 was released on 22 September 2026 at \$4 per million input tokens and \$20 per million output tokens, with prompt cache reads at \$0.20 per million. Context is up to 1M tokens; maximum output is 128K, with 300K available on the Batch API behind a beta header.

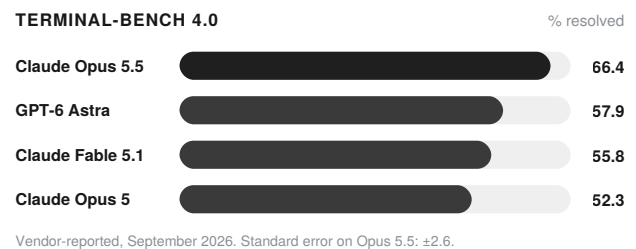


Fig. 4. Terminal-Bench 4.0, vendor-reported. The gap that matters for a company brain is not the headline number but what it costs to reach it.

Anthropic reports roughly 30% faster output and about 40% lower cost per task than Opus 5, with the token-savings component depending on effort settings and workload. The published examples are the kind of work a company brain is for: a 680,000-line code migration finished in under a day; a 200,000-line audit in under three hours against more than twenty for Opus 5; a HAProxy rewrite from C to Rust in 9.5 hours against 12 for Fable 5.1, at 51% lower cost.

TABLE III
CLAUDE OPUS 5.5, PUBLISHED FIGURES

Metric	Value
Terminal-Bench 4.0	66.4% (SE ± 2.6)
FrontierCode v1.1	54.4%
CursorBench 4.0	57.8%
OSWorld 2.0, partial pass	81.8%
AutomationBench	40.0%
Humanity's Last Exam, with tools	67.7%
GDPval-AA v2.1	1846 Elo
Context / max output	1M / 128K tokens
Price	\$4 in, \$20 out, \$0.20 cache read

A. Why the cache price is the interesting line

The headline benchmarks get the attention; the \$0.20 cache read is what changes architecture. A cached prefix is twenty times cheaper to re-read than a fresh one. That single ratio makes one long-lived, warm context dramatically cheaper than a pattern that keeps tearing context down and rebuilding it somewhere else. Hold that thought until Section VII, because it is the reason naive routing loses.

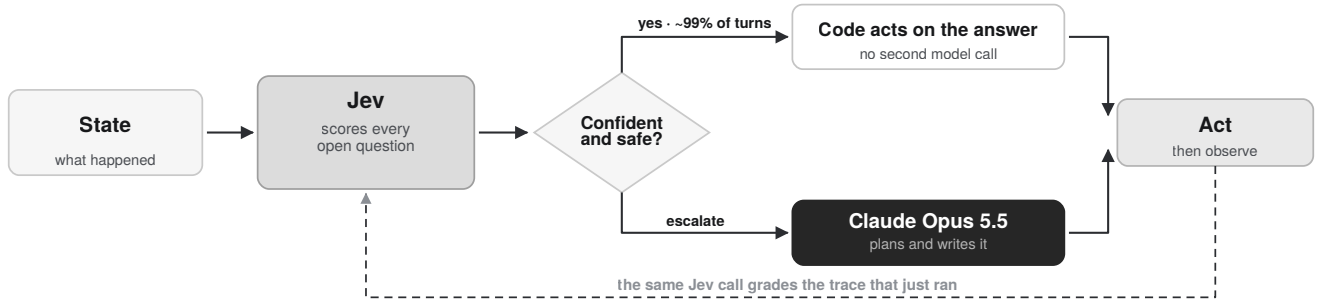


Fig. 5. One turn through the brain. The escalation branch is the only one that spends frontier tokens, and the same primitive that made the decision also grades the result.

C. Context assembly

The decision layer produces a relevance score per chunk, and context assembly uses it. In its simplest

V. THE COMPANY BRAIN

Fig. 1 is the architecture this paper argues for. It has six parts, and only two of them are models.

A. Shared state

Everything the company already produces, addressable: tickets and email, the repo and its open pull requests, agent traces, docs and wiki pages, CRM records, and the cached prefix from the last turn. The important design choice is that state is explicit and chunked rather than implicit in a growing conversation. If every piece of context is an addressable, typed chunk, it can be scored individually. If it is a blob of chat history, it can only be compacted, and compaction is lossy compression performed without knowing what the next query will be.

B. The decision layer

A bank of typed questions asked against that state on every turn. The six in Fig. 1 are a reasonable starting set: is it urgent, how hard is it, does it touch secrets, which model should answer, which tool should run, is this command allowed. They are asked together, in one request, and they come back as numbers.

form that is a Noul on every chunk of context: tool-call inputs, tool-call outputs, internal reasoning, even earlier turns with the user. A richer version is a Score with four

levels — do not show, show a short summary, show a longer summary, show the whole thing. The consequence is that context stops being a static thing you append to and becomes something recomputed per query, cheaply.

D. The router

The router takes difficulty, risk and cost and picks a model. LangChain ships this as middleware: you declare named choices with criteria and an instruction like "choose the least costly model that can complete the task", and Jev assesses the request. The probabilities and confidence stay in agent state afterwards, which matters more than it sounds — you can log why a turn was routed the way it was and audit the distribution later.

```
from langchain.agents import create_agent
from langchain_typesafe.experimental import (
    middleware as ts)

router = ts.ModelRouterMiddleware(
    choices={
        "fast": ts.ModelChoice(
            model="anthropic:claude-haiku-4-5",
            criteria="Lookups, local edits."),
        "deep": ts.ModelChoice(
            model="anthropic:claude-opus-5-5",
            criteria="Architecture, migrations."),
    },
    instructions="Pick the least costly model "
               "that can finish the task.")

agent = create_agent(
    "anthropic:claude-haiku-4-5",
    middleware=[router])
```

E. Runtime and tools

The runtime is the loop: act, observe, repeat. Below it sits the tool surface, and this is where the second structural idea lives. Tools normally have to be declared up front in the system message, which burns context on capabilities that are irrelevant most of the time and makes the model worse at choosing among them. The alternative is a middle layer: short snippets describing what exists, with the full schema loaded only when a decision layer says that tool is in play. If the schema is only paid for when it is used, a hundred tools and a thousand documents can ship by default.

VI. SIX WORKFLOWS THAT PAY FOR THEMSELVES

SIX PLACES A TYPED DECISION REPLACES A CHAT CALL

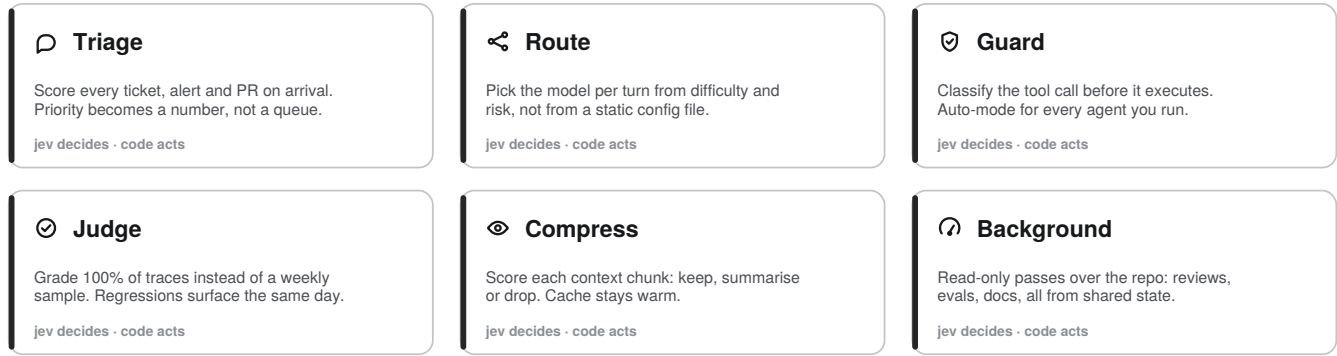


Fig. 6. Six places inside an existing system where a typed decision replaces a chat call. None of them requires rewriting the agent.

The architecture is easier to adopt than it looks, because it decomposes. Each of the six below is independently useful and independently reversible.

Triage. Every ticket, alert and pull request gets scored on arrival: urgency, difficulty, blast radius, which team. Priority stops being a queue position someone set by hand and becomes a number with a distribution behind it.

Routing. Model choice per turn instead of per deployment. The important part is not saving money on the easy turns; it is that hard turns reliably reach the model that can finish them, instead of burning three cheap attempts first.

Guarding. Coding harnesses such as Claude Code, Codex and Cursor already classify dangerous actions before taking them, and that step is what made people willing to leave them running. Until recently that classifier lived inside closed source. With a cheap, fast classifier available, the same pattern can be applied to every agent a company runs.

Judging. Grade every trace instead of sampling. Section IX has the numbers that make this affordable.

Compressing. Score each chunk rather than compacting the whole conversation. Keep the cache warm; drop what stopped mattering.

Background work. Read-only passes over the current state of the repo: cross-model review, generated evals, documentation, explanations. These are cheap to run in parallel precisely because they share the same scored state, so the expensive part — working out what is relevant to the current change — is paid for once.

VII. ROUTING ECONOMICS, HONESTLY

Here is where most designs go wrong, and the arithmetic is worth doing in public. The intuition is that routing an easy turn to a cheaper model saves money. In a long session it frequently does not, because the expensive model has to re-read everything the cheap model produced in order to take over.

Take the worked example from the TypeSafe note. Let X be context tokens loaded, Y output tokens generated, and Z additional tokens generated inside that output such as commands and file reads, with a plausible session mix of $X = 0.65$, $Y = 0.12$, $Z = 0.23$. Compare a pure Opus 5 path against Opus 5 → Sonnet → Opus 5:

pure	= $25*Y + 5*Z$	= 4.15
routed	= $3*X + 15*Y + 3*Z$	(cheap)
	+ $5*(Y + Z)$	(reload)
		= 6.19

Routing down and back costs about 50% more, because the large model pays to load the context it

skipped. Re-run the same structure at Opus 5.5 prices and the conclusion survives: 3.32 against 5.84.

COST OF ONE SESSION, NORMALISED



Same workload mix (X=0.65 context, Y=0.12 output, Z=0.23 read) applied to published list prices. Routing down and back costs more because the large model has to re-read the context it skipped.

Fig. 7. Normalised session cost. Routing down and back is worse in both generations. The bottom bar is the only pattern that wins, and it wins by not moving the context at all.

So what does work? Two things, and neither is "route to a cheaper LLM".

The first is to keep one context warm and pay the cache price to re-read it. At \$0.20 per million, re-reading the prefix is twenty times cheaper than loading it fresh, and the reload tax that ruins the routed path largely disappears.

The second, and the real one, is to remove the model from the decision entirely. A Jev call does not load your session context into a frontier model. It evaluates a state and returns a value for a third of a cent. The bottom bar in Fig. 7 is not cheaper because it found a discount tier; it is cheaper because roughly ninety-nine percent of the turns never became a generation call in the first place.

RULE OF THUMB. Route work, not context. Moving a task between models is expensive. Replacing a decision with a typed answer is not.

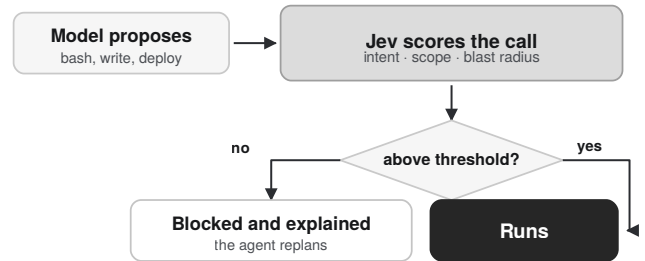
TABLE IV
WHICH TIER SHOULD ANSWER

The question in front of you	Answer with
Is this urgent, risky, in scope?	Jev
Which of these five tools applies?	Jev
Is this chunk still worth carrying?	Jev
Was that trace good enough to pass?	Jev
Should this command be allowed to run?	Jev
Why is this test flaky?	Opus 5.5
Write the migration and explain it	Opus 5.5
Plan a refactor across 40 files	Opus 5.5
Summarise this incident for the board	Opus 5.5

VIII. THE TRUST BOUNDARY

Agents are still inherently untrustworthy. An agent can receive bad instructions — naturally, or from a motivated attacker — that persuade it into taking an action nobody wanted. The mitigation that actually shipped in practice was not better prompting. It was a classifier in front of the action.

THE GATE EVERY TOOL CALL PASSES



Same primitive as a closed coding harness, but in your own code.

Fig. 8. The gate. A model proposes; a typed decision decides whether it executes. The agent never learns that it was blocked by argument, only that the call failed.

LangChain exposes this as auto-mode middleware. You name the tools you want gated, and Jev checks the call for risky decisions before the tool executes:

```

from langchain.agents import create_agent
from langchain_typesafe.experimental import (
    middleware as ts)

guardrail =
ts.AutoModeMiddleware(tools=["bash"])
agent = create_agent(
    "anthropic:claude-opus-5-5",
    middleware=[guardrail])

```

Three design notes matter more than the three lines of code. First, the gate runs on the *call*, not on the conversation, so a model that has been talked into something still has to produce a concrete command that passes. Second, because the check is cheap, it can be layered: intent, scope, blast radius, and budget as four separate Nouns rather than one overloaded judgement. Third, the refusal should be legible — log the probability and the question that failed, or you will spend a week arguing with a number nobody can explain.

A. Security-aware routing

There is a version of routing that has nothing to do with cost. Cheap models hosted in jurisdictions you do not control are attractive for bulk work and unattractive for anything touching secrets. If a decision layer can

estimate the likelihood that a sub-task will read a particular class of file, you can write policy against that estimate: this class of work may go anywhere, that class stays on a specific provider. Difficulty is only one axis. Data class is another, and it is the one auditors ask about.

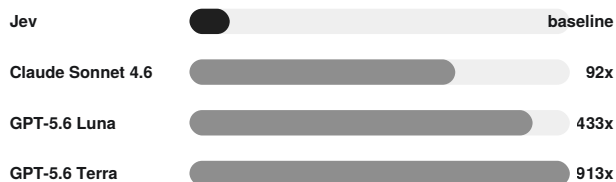
IX. EVALUATION: THE JUDGE GETS CHEAP

Agent evaluation has had two options, and both have a tax. Code-based evaluators are cheap, fast and reliable, but they need deterministic inputs, so they struggle the moment an open-ended task has several valid answers. LLM-as-judge handles the open-ended case but is slow, expensive and non-deterministic — a shaky foundation for a testing apparatus.

Evaluation is, structurally, a decision task: given a state and a behaviour, assign a score. LangChain ran the obvious experiment. They built a weather agent, captured five runs as fixed examples, had a human label each one, and then replayed them one hundred times per judge against Jev, GPT-5.6 Luna, GPT-5.6 Terra and Claude Sonnet 4.6.

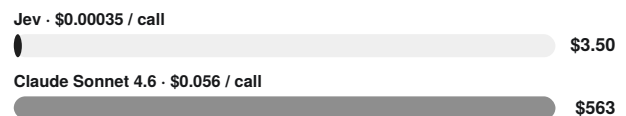
A. SCORE VARIANCE ON IDENTICAL BEHAVIOUR

relative to Jev, lower is better · log scale



B. DAILY COST OF JUDGING 10,000 TRACES

derived from observed per-call cost in the same experiment



Same experiment: Jev matched the human label on all 500 decisions

Sonnet 4.6 matched on 80.0%, Luna 96.4%, Terra 99.8%

Fig. 9. Results of LangChain's judge comparison. Left: observed score variance relative to Jev on identical agent behaviour. Right: daily cost of judging 10,000 traces, derived from the observed per-call cost.

On the binary pass/fail decision, Jev matched the human oracle on all five hundred repeated decisions. Terra matched on 99.8%, Luna on 96.4%, Claude Sonnet 4.6 on 80.0%. On the continuous quality score,

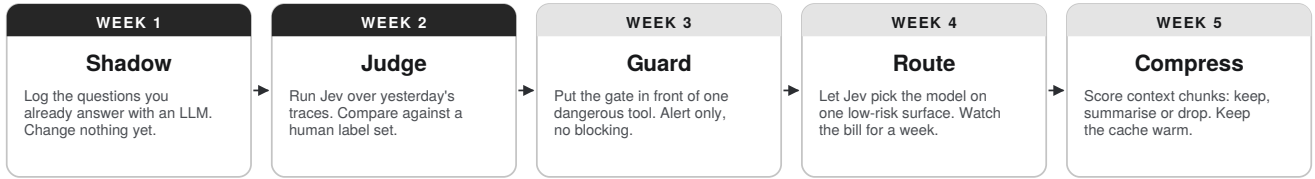
Jev had the lowest observed mean per-case variance at 0.0000149; Luna was 433x higher, Terra 913x higher, Claude 92x higher. Jev averaged 0.44 s and \$0.00035 per call, \$0.34 in total against \$28.17 for Claude.

TABLE V
JUDGE COMPARISON, AS REPORTED

Judge	Oracle match	Variance vs Jev	Total cost
Jev	100.0%	baseline	\$0.34
GPT-5.6 Terra	99.8%	913x higher	—
GPT-5.6 Luna	96.4%	433x higher	—
Claude Sonnet 4.6	80.0%	92x higher	\$28.17

Two caveats belong next to those numbers, and LangChain states both. The test is narrow: five examples, one agent, one domain. And lower variance is not automatically higher accuracy — a judge can be consistently wrong, and cheap evaluation amplifies a wrong judge exactly as efficiently as a right one. Human review and judge alignment do not go away; they get cheaper to run against.

A FIVE-WEEK ROLLOUT THAT DOES NOT TOUCH PRODUCTION FIRST



Nothing is replaced. A typed decision is added beside the call that already exists, then the call is removed once the numbers agree.

Fig. 10. Five weeks, in order of reversibility. Nothing in weeks one to three can break production, because nothing in weeks one to three is allowed to change an outcome.

Week 1, shadow. Find the places where you already ask a model a closed question and log them. Do not change behaviour. You are building a corpus of the decisions your system actually makes, which you almost certainly do not have.

Week 2, judge. Run Jev over yesterday's traces and compare against a human-labelled set. This is the lowest-risk surface in the whole system: a judge that is wrong costs you a bad dashboard, not a bad deploy.

Week 3, guard. Put the gate in front of exactly one dangerous tool, in alert-only mode. Watch what it

What changes is the budget line. At roughly three and a half dollars a day to grade ten thousand production traces, the tradeoff between coverage and cost mostly disappears. You stop sampling. You grade every run, against several focused criteria rather than one overloaded rubric, and you repeat the judgement when confidence is low. The unlock is not cheaper evals; it is a feedback loop tight enough that a regression is visible the same day it ships.

X. A FIVE-WEEK ROLLOUT

The failure mode when adopting this is to rip out a working chat call and replace it with a typed one on faith. Do the opposite: add the typed decision next to the call that already exists, compare them, and only then delete.

would have blocked for a week before you let it block anything.

Week 4, route. Let the decision layer pick the model on one low-risk surface and watch the bill and the failure rate together. Section VII is the warning: if the numbers get worse, you are moving context, not work.

Week 5, compress. Score context chunks and drop what stopped mattering. Do this last, because it is the change most likely to produce a subtle quality regression that only shows up three turns later.

XI. ANTI-PATTERNS

TABLE VI
SEVEN WAYS THIS GOES WRONG

Anti-pattern	Why it bites
Prompting Jev like a chat model	There is no text output to prompt for. Write questions, not instructions.
One overloaded question	"Rate this run" hides four judgements. Ask four Nouns and keep them separately auditable.
Hard thresholds, no logging	A bare 0.5 cutoff with nothing recorded is unarguable in a postmortem. Log the question and the probability.
Routing to save money	Moving context between models costs more than it saves. Remove decisions instead.
Cheap evals, no humans	A consistently wrong judge at \$0.00035 produces wrong feedback very efficiently.
Gating the conversation	Gate the concrete tool call. Intent can be talked around; a command cannot.
Compressing before measuring	Dropping the wrong chunk fails three turns later, where nobody is looking.

XII. LIMITATIONS

This architecture is not free of assumptions, and they should be stated plainly.

The performance claims for Jev are vendor claims. Up to 200x faster inference and 400x lower cost than comparable LLMs on classification tasks is TypeSafe's figure, on TypeSafe's framing of the task. The independent numbers in this paper come from a single narrow experiment run by a party with a commercial integration.

Calibration is load-bearing. Everything above assumes the probability means what it says. When a decision layer is miscalibrated, you do not get a loud failure; you get a system that is confidently wrong at scale, quietly, and the cheapness that made it attractive is exactly what makes the blast radius large.

Explicit state is work. The design assumes chunked, addressable, scoreable context. Most existing agents do

not have that; they have a conversation. Retrofitting it is the real migration cost, not the API call.

And the cost of the frontier tier is not zero after all this. It is concentrated. A company brain does not make Opus 5.5 cheap; it makes sure that when you pay for Opus 5.5, you are paying for thinking rather than for plumbing.

XIII. CONCLUSION

Generation stopped being the bottleneck. Deciding what to generate, what to read, what to keep and what to let run is now the expensive part, and until recently there was no correctly shaped tool for it — only a text generator being asked to behave like a classifier.

The pairing in this paper is deliberate and it is not a marketing line. Claude Opus 5.5 is the best version yet of a system that thinks: a million tokens of context, benchmark-leading agentic performance, and a price that finally makes long-horizon work defensible. Jev is a system that decides: typed, parallel, calibrated, and cheap enough to put in front of everything. Wired together by ordinary code that holds the branch, they are the closest thing on offer to a brain that a company can actually run — one that spends almost nothing to notice, and spends properly to think.

ACKNOWLEDGMENT AND SOURCES

This document is an independent synthesis prepared for study. It is not affiliated with, reviewed by, or endorsed by TypeSafe AI, LangChain or Anthropic. All diagrams are original work produced for this paper; no figure is reproduced from a source. Quoted metrics retain the framing of their source and are attributed below.

- [1] LangChain, "Jev: a System One model in the agent loop," integration post, 2026. Source for the System One definition, the three question types, parallel question evaluation, TypeSafeClassifier, ModelRouterMiddleware,

AutoModeMiddleware, and the support-ticket request/response example.

- [2] LangChain, "Evaluating agents with Jev," experiment write-up, 2026. Source for the judge comparison: oracle agreement, per-case variance, 0.44 s average latency, \$0.00035 per call, \$0.34 against \$28.17, and the reproducibility notes.
- [3] TypeSafe AI, product documentation and Quickstart. Source for question semantics and the "jev-latest" request shape.
- [4] TypeSafe AI, engineering note on a typesafe-centric coding agent, 2026. Source for the routing arithmetic, the KV-cache framing, meta-attention over context chunks, snippet-first tool exposure, security-aware routing, and the background-processing pattern.
- [5] Anthropic, Claude Opus 5.5 release, 22 September 2026. Source for pricing (\$4 / \$20 / \$0.20 cache read), 1M context, 128K output, ~30% faster, ~40% cheaper per task, and the migration, audit and HAProxy examples.
- [6] Vellum, "Claude Opus 5.5 Benchmarks Explained," 2026; Digital Applied, "Claude Opus 5.5: Pricing, Benchmarks and Breaking Changes," 2026. Sources for the benchmark table and the Terminal-Bench 4.0 comparison set.
- [7] Microsoft, fastcontext; and the tooling surveyed in [4] (ast-grep, rtk, headroom, fff) for the tool-surface discussion.

Derived figures are labelled in place. The daily cost of judging 10,000 traces in Fig. 9 is arithmetic on the per-call costs reported in [2], not a measurement. The Opus 5.5 row in Fig. 7 applies the cost structure from [4] to the list prices in [5].

APPENDIX A: GLOSSARY

TABLE VII
TERMS USED IN THIS PAPER

Term	Meaning
System One model	A model built to return typed answers and probabilities for a state, rather than to generate text.
RLCD	Reinforcement learning for calibrated decisions; the training approach TypeSafe cites for Jev.
Noul	A yes/no question type returning the probability that a statement is true.
Choice	A question type selecting one option from a set, with per-option probabilities and a confidence.
Score	A question type rating a state against an ordered rubric, returning a continuous value.
Calibration	The property that a stated probability matches observed frequency. The assumption everything else rests on.
Decision layer	The bank of typed questions asked against shared state on every turn.
Context assembly	Rebuilding the working context per query from scored chunks, rather than appending to a transcript.
Auto-mode	Classifying a proposed tool call and blocking it before execution.
Escalation	The branch where a typed decision hands a turn to the frontier tier.
Reload tax	The cost the large model pays to re-read context that a smaller model produced.
Online evaluation	Grading production traces continuously rather than sampling them after the fact.

APPENDIX B: A STARTER QUESTION LIBRARY

Twelve questions worth asking on day one. Each is atomic, each is independently loggable, and all twelve fit in a single request.

TABLE VIII
TWELVE QUESTIONS TO ASK EVERY TURN

Name	Type	Asks
is_urgent	noul	does this need attention right now
difficulty	score	1 trivial to 5 architectural
touches_secrets	noul	would this read a credential or key
is_destructive	noul	could this delete or overwrite state
in_scope	noul	is this inside what the agent was asked to do
best_tool	choice	which tool applies, or none
needs_frontier	noul	does this turn need the top tier
chunk_relevance	score	keep whole, summarise, or drop
cache_still_good	noul	is the existing prefix worth re-using
grounded	noul	is the answer supported by retrieved evidence
answer_quality	score	1 unhelpful to 5 highly useful
did_pass	noul	did this run meet the bar

The first six run before the turn. The next three shape it. The last three grade it afterwards, against the same state, using the same primitive. That symmetry is the quiet advantage of the whole design: the thing that decided what to do is the same thing that decides whether it worked.

APPENDIX C: BUILD CHECKLIST

Ten checks to run against a design before it ships. Each one corresponds to a failure this paper has described.

TABLE IX
TEN CHECKS BEFORE YOU SHIP

#	Check	Pass condition
1	Is every decision in the loop named?	You can list them. If you cannot, the prompt is doing work nobody reviewed.
2	Is each one atomic?	One question, one answer, one log line. No composite rubrics.
3	Is the probability logged with the branch?	A postmortem can reconstruct why the system chose what it chose.
4	Is there a human-labelled set?	At least fifty examples per question, refreshed quarterly.
5	Is the gate on the call, not the intent?	The concrete command is what gets classified.
6	Is the frontier tier reached only on escalation?	Measure the escalation rate. If it is above ten percent, the questions are wrong.
7	Is the context cached rather than rebuilt?	Cache-read tokens dominate fresh input tokens in the bill.
8	Does routing actually reduce cost?	Compare against a pure frontier baseline for a week. If not, stop routing.
9	Is every trace judged?	Coverage is 100%, not a sample, and the judge itself is audited.
10	Can you turn it all off?	Every typed decision has a documented fallback to the call it replaced.

The last check is the one teams skip and the one that matters most. A decision layer is infrastructure. Infrastructure that cannot be bypassed during an incident is not infrastructure; it is a dependency. Keep the old call path behind a flag until the numbers have been boring for a month.